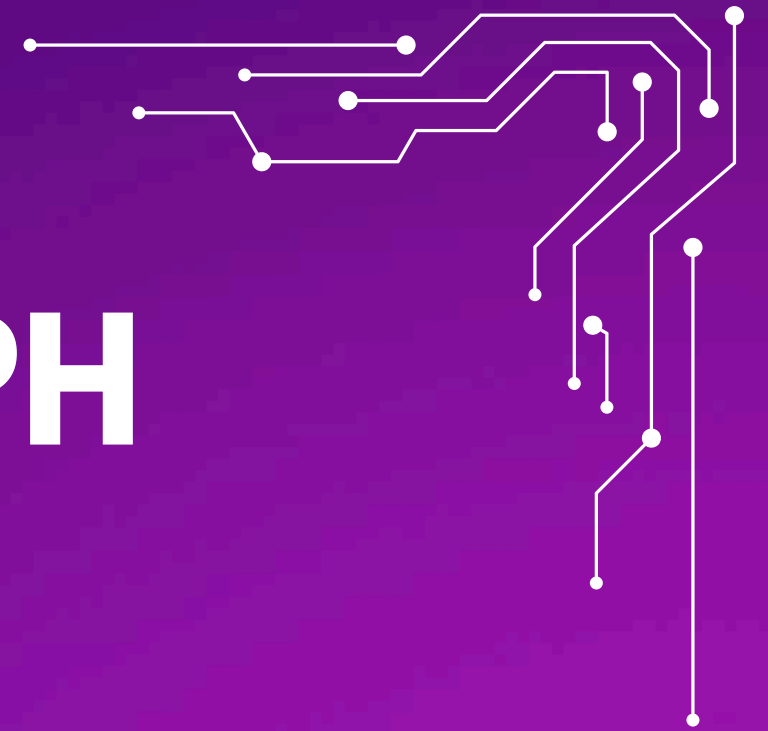
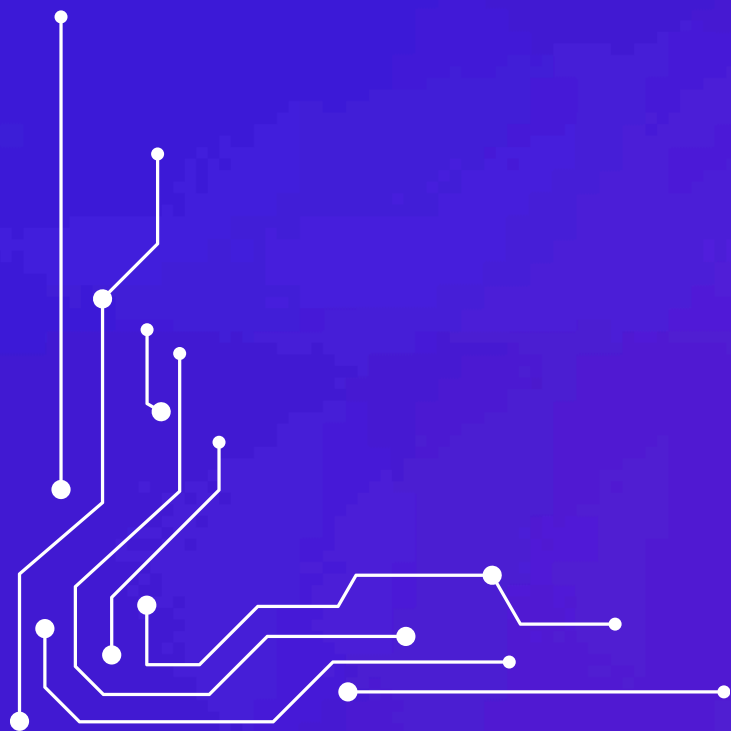


INTRODUCTION TO GRAPH THEORY AND TYPES OF GRAPHS





WHAT IS GRAPH THEORY

Graph Theory is a branch of mathematics that studies relationships between objects.

Composition of a graph:

A graph consists of:

Vertices (Nodes): represent objects

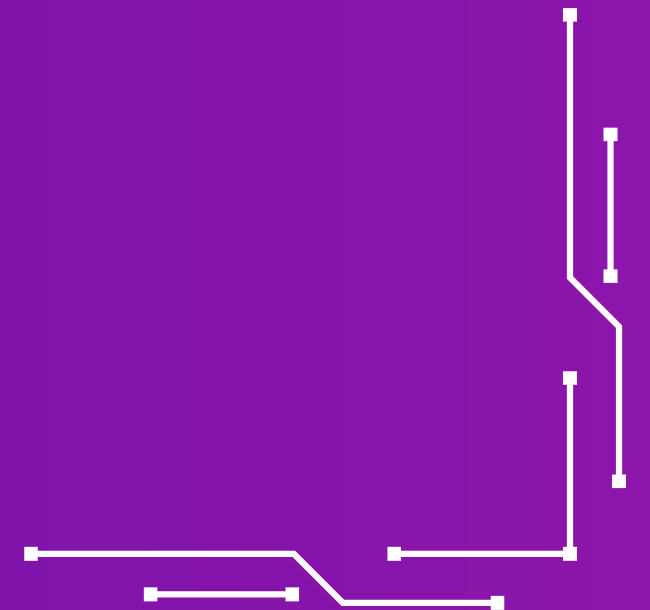
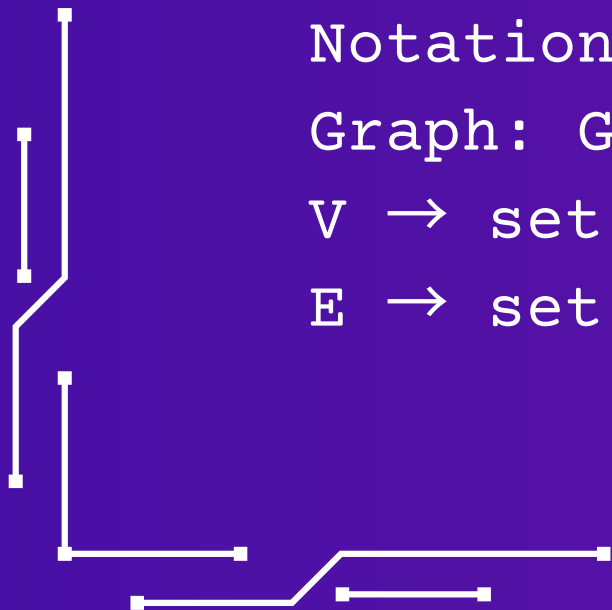
Edges (Links): represent connections

Notation:

Graph: $G = (V, E)$

$V \rightarrow$ set of vertices

$E \rightarrow$ set of edges



TYPES OF GRAPHS

...

Graphs can be classified into different groups:

Based on Direction

- Undirected
- Directed

Based on Edge Weights

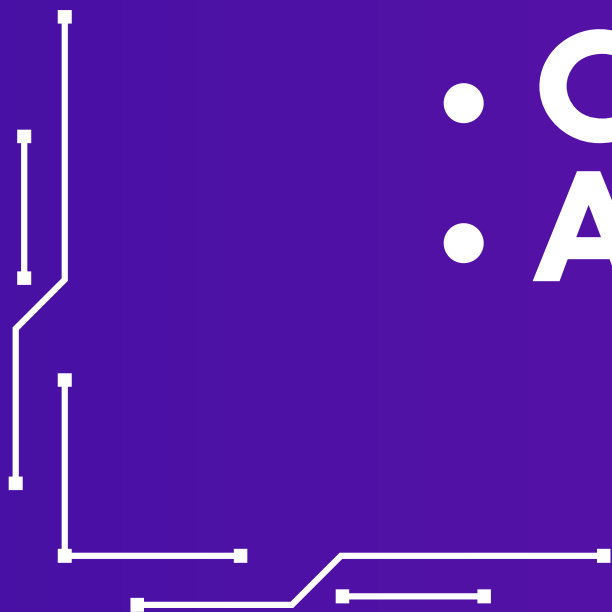
- Weighted
- Unweighted

Based on cycles

- Cyclic
- Acyclic

Based on Connectivity

- Connected
- Disconnected



...

REAL LIFE EXAMPLES OF GRAPH NETWORKS

1. Social Media Networks

- Users can be represented by vertices (nodes)
- Friend / follow relationship can be represented by edges
- Example: Instagram, Facebook

2. Road and Transportation Networks

- Cities can be represented by vertices
- Roads can be represented by edges
- Used in GPS and shortest path algorithms



DEPTH-FIRST SEARCH



02

DEPTH-FIRST SEARCH (DFS): EXPLORING THE EXTREMES

- How it Works: Depth-First Search (DFS) explores a path fully before retracing steps, similar to driving down a highway until reaching a dead end, then exploring the side roads.
- Data Structure Used: DFS uses recursion (call stack) or an explicit stack data structure to keep track of vertices being explored.
- Application: It traverses adjacency lists to count connected components and identify isolated clusters within the 500-node city network.
- Performance:
Time Complexity: $O(V + E)$
Space Complexity: $O(V)$



03



0 → 1, 2
 1 → 0, 3, 4
 2 → 0, 5
 3 → 1
 4 → 1
 5 → 2

DFS Traversal: 0 1 3 4 2 5

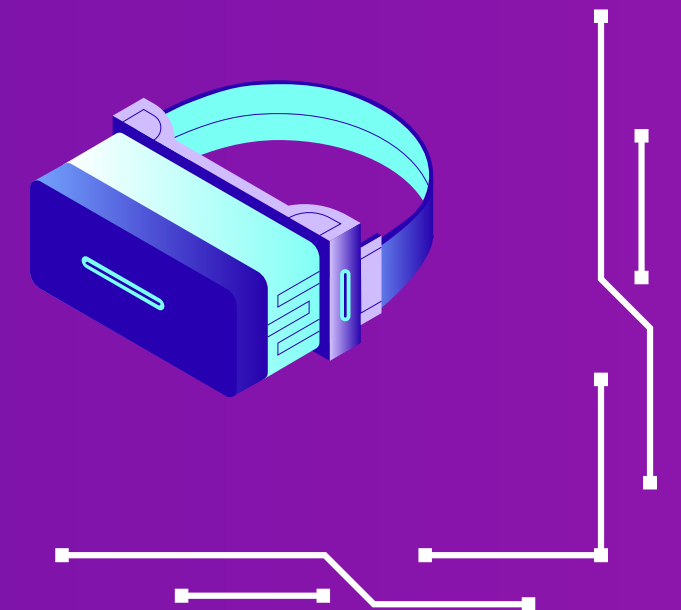
DFS CODE

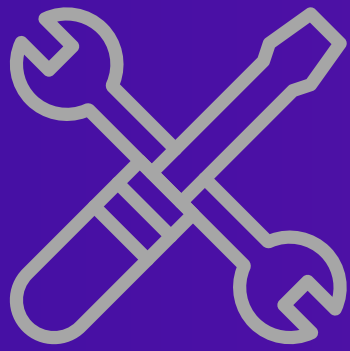
```

void dfs(int node,vector<vector<int>>& adj,vector<int>& vis,vector<int>& dfsarr){
    vis[node]=1;
    dfsarr.push_back(node);
    //traverse through all its neighbours
    for(auto it:adj[node]){
        if(!vis[it]){
            dfs(it,adj,vis,dfsarr);
        }
    }
}
  
```

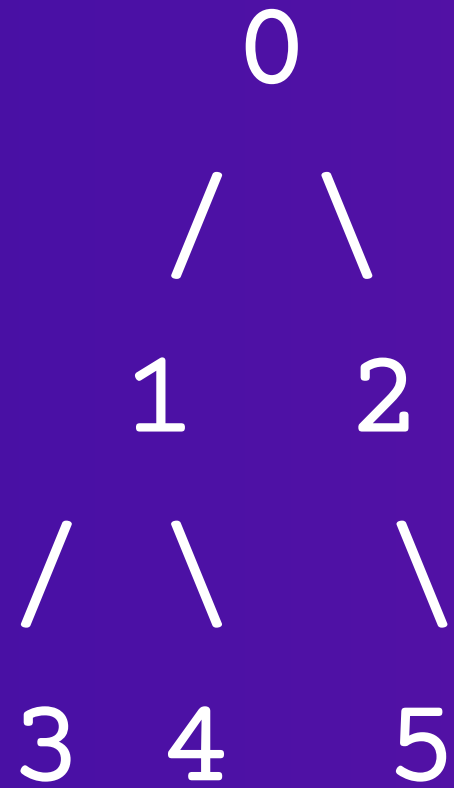
```

vector<int> dfsOfGraph(int V, vector<vector<int>> edges) {
    int n=edges.size();
    vector<vector<int>> adj(V);
    for(int i=0;i<n;i++){
        int u=edges[i][0];
        int v=edges[i][1];
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    vector<int> dfsarr;
    int start=0;
    vector<int> vis(V);
    dfs(start,adj,vis,dfsarr);
    return dfsarr;
}
  
```





DFS WORKING (DRY RUN)



Start DFS from node 0

Step	Current Node	Action	Visited	DFS Array
1	0	Visit 0	{0}	[0]
2	1	Go to unvisited neighbour 1	{0,1}	[0, 1]
3	3	Go to unvisited neighbour 3	{0,1,3}	[0, 1, 3]
4	3	No unvisited neighbours, backtrack to 1	{0,1,3}	[0, 1, 3]
5	4	Go to next unvisited neighbour 4	{0,1,3,4}	[0, 1, 3, 4]
6	4	No unvisited neighbours, backtrack to 0	{0,1,3,4}	[0, 1, 3, 4]
7	2	Go to next unvisited neighbour 2	{0,1,3,4,2}	[0, 1, 3, 4, 2]
8	5	Go to unvisited neighbour 5	{0,1,3,4,2,5}	[0, 1, 3, 4, 2, 5]
9	5	No unvisited neighbours, backtrack to 2 then 0 (end)	{0,1,3,4,2,5}	[0, 1, 3, 4, 2, 5]

Backtracking Explained: When DFS reaches a node that has no unvisited neighbours, it backtracks to the previous node and continues with the next unvisited neighbour. This process continues until all nodes are visited.



BREADTH-FIRST SEARCH



BREADTH-FIRST SEARCH (BFS): EXPANDING THE PERIMETER

- How It Works: Breadth-First Search (BFS) explores the network layer by layer, much like a radio signal that broadcasts from a city to its neighboring areas, then to those neighbors' neighbors, and so forth.
- Application: BFS is used to find the largest connected component by beginning from a start node and logging all reachable cities.
- Performance:
- Time Complexity: $O(V + E)$
- Space Complexity: $O(V)$

This method offers an efficient way to traverse and analyze network structures.



06

BFS CODE & OUTPUT SNAPSHOT

```
1 // BFS traversal for connectivity analysis
2 int BFS(int startCity, vector<bool>& visited, vector<vector<int>>& adj) {
3     queue<int> q;
4     q.push(startCity);
5     visited[startCity] = true;
6     int componentSize = 0;
7
8     while (!q.empty()) {
9         int curr = q.front();
10        q.pop();
11        componentSize++;
12
13        for (int neighbor : adj[curr]) {
14            if (!visited[neighbor]) {
15                visited[neighbor] = true;
16                q.push(neighbor);
17            }
18        }
19    }
20    return componentSize;
21 }
22
```



Dijkstra's Algorithm

- >Dijkstra's Algorithm is a greedy shortest-path algorithm.
- >It finds the minimum distance from a source node to all other nodes in a weighted graph.
- >Works only when all edge weights are non-negative.

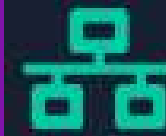
Applications:

- 1.GPS and navigation systems
- 2.Network routing protocols
- 3.Transportation and logistics planning
- 4.Robot path planning
- 5.Social network analysis



GPS & Navigation

Google Maps, Waze — fastest route calculation



Network Routing

OSPF protocol finds optimal data packet paths



Game AI

Pathfinding for NPCs in open-world games



Logistics & Delivery

Minimise travel cost for delivery fleets

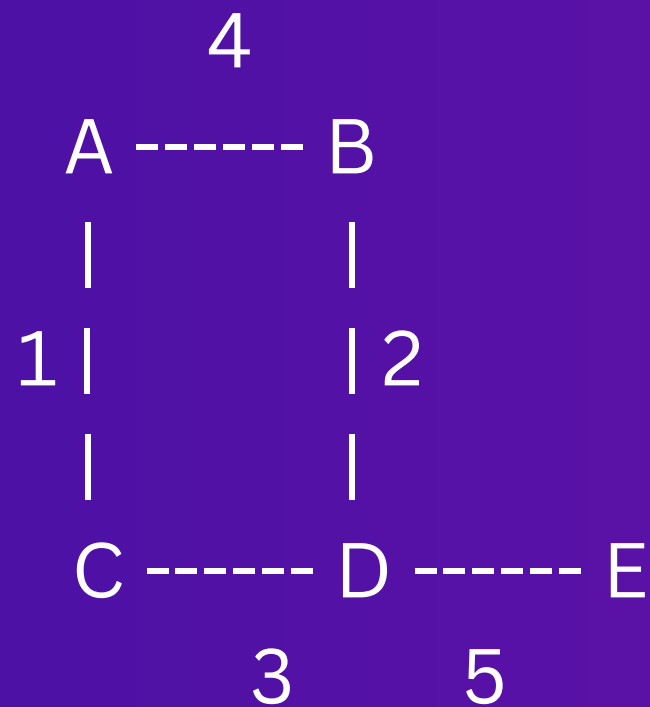
COMPLEXITY

Time :- $O((V+E) \log V)$ with
Priority Queue

Space :- $O(V)$ for
dist & Priority
Queue

```
vector<int> dijkstra(int V, vector<vector<int>> adj[], int S) {  
    priority_queue<pair<int,int>,  
        vector<pair<int,int>>,  
        greater<pair<int,int>>> pq;  
  
    vector<int> dist(V, 1e9);  
    dist[S] = 0;  
    pq.push({0, S});  
  
    while (!pq.empty()) {  
        auto [d, node] = pq.top();  
        pq.pop();  
  
        for (auto &it : adj[node]) {  
            int adjNode = it[0], wt = it[1];  
  
            if (d + wt < dist[adjNode]) {  
                dist[adjNode] = d + wt;  
                pq.push({dist[adjNode], adjNode});  
            }  
        }  
    }  
    return dist;  
}
```

Example Graph



Source Vertex: A

Final Shortest Distances from A:

- A = 0
- B = 4
- C = 1
- D = 4
- E = 9

Iterations

STEP	SELECTED NODE	DISTANCE ARRAY
INITIAL	-	(0, ∞, ∞, ∞, ∞)
1	A	(0, 4, 1, ∞, ∞)
2	C	(0, 4, 1, 4, ∞)
3	B	(0, 4, 1, 4, ∞)
4	D	(0, 4, 1, 4, 9)
5	E	(0, 4, 1, 4, 9)

SMART TRANSPORTATION NETWORK DESIGN

USING GRAPH THEORY AND ROUTE
OPTIMISATION

PURPOSE

THE GOAL OF THIS PROJECT IS TO BUILD A PROTOTYPE OF A SMART TRANSPORTATION NETWORK AND IDENTIFY THE ISOLATED AND CONNECTED CITIES AND FORM CLUSTERS OF CITIES IN CLOSE PROXIMITY TO EACH OTHER. FURTHER, WE ALSO CALCULATE THE SHORTEST PATHS AND DISTANCES FROM ANY CITY TO ANY OTHER REACHABLE CITY ON THE NETWORK.



STUDY OBJECTIVES

DATA GENERATION

USE OF THE RAND AND
RANDS MODULES TO
GENERATE THE NETWORK
ADJACENCY LIST WITH
RANDOMLY GENERATED
WEIGHTS AND
CONNECTIONS

CONNECTIVITY AND CLUSTERING

USE OF DFS OR BFS TO
FIND THE LIST OF
CONNECTED CITIES AND
ORGANISE THEM INTO
CLUSTERS

PATH GENERATION AND SHORTEST DISTANCE

USE OF DJIKSTRA'S
ALGORITHM TO FIND THE
SHORTEST DISTANCE
BETWEEN ANY TWO
CITIES OF THE
WEIGHTED NETWORK AND
RETURN INF IF
UNREACHABLE

DATA GENERATION

WE CHECK PAIRS OF NODES AND ASSIGN THEM A PROBABILITY WHETHER THEY'RE CONNECTED OR NOT. A HIGHER PROBABILITY LEADS TO MORE GRAPH CONNECTIONS AND LESSER ISOLATED NODES.

IF THE PAIR OF CITIES IS CONNECTED, WE ASSIGN A RANDOMLY GENERATED NUMBER BETWEEN TWO NUMBERS OF OUR CHOOSING TO BE THE DISTANCE BETWEEN THEM

WE ALSO HAVE TO ENSURE THE EDGES ARE BIDIRECTIONAL (I.E IF CITY A IS CONNECTED TO CITY B, CITY B IS ALSO CONNECTED TO CITY A)

DEPTH FIRST SEARCH

DFS EXPLORES EACH NODE TO THE MAX DEPTH FIRST BEFORE JUMPING TO ANY MORE NODES

TIME COMPLEXITY: $O(V+E)$

SPACE COMPLEXITY: $O(V)$

BREADTH FIRST SEARCH

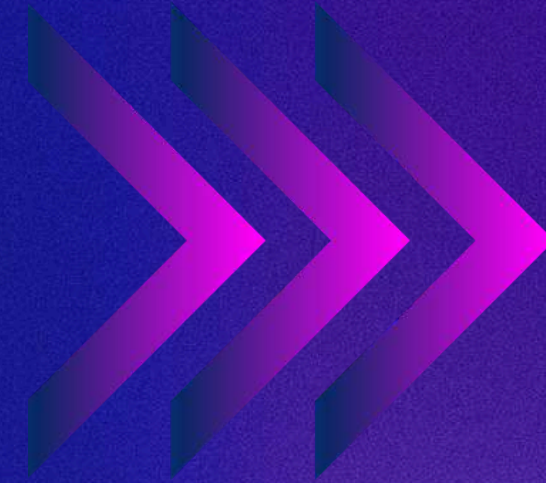
BFS EXPLORES EVERY NODE OF THE SAME LEVEL FIRST AND THEN PROCEEDS LAYER BY LAYER SEQUENTIALLY

TIME COMPLEXITY: $O(V+E)$

SPACE COMPLEXITY: $O(V)$

CLUSTERING METHODS

DJIKSTRA'S ALGORITHM



WE USE DJIKSTRA'S ALGORITHM TO CALCULATE THE SHORTEST DISTANCE BETWEEN ANY TWO CONNECTED NODES EFFICIENTLY. IF WE FIND THAT THERE IS NO PATH WHICH CONNECTS THEM, WE ASSIGN THE DISTANCE BETWEEN THEM TO BE INFINITY. WE NOW USE THE ADJACENCY LIST GENERATED IN THE DATA GENERATION STEP AND FEED IT TO THE ALGORITHM TO WORK WITH.



TIME AND SPACE COMPLEXITIES

DATA GENERATION

TIME EFFICIENCY:
 $O(V+E)$
SPACE EFFICIENCY:
 $O(V+E)$

CONNECTIVITY

TIME EFFICIENCY:
 $O(V+E)$
SPACE EFFICIENCY:
 $O(V)$

SHORTEST PATH

TIME EFFICIENCY:
 $O(E \log V)$
SPACE EFFICIENCY:
 $O(V)$

IMPLEMENTATION



CREDITS

THANKS TO EVERYONE WHO MADE THIS POSSIBLE:

INTRO: KUNAL K, ANSHU S, MAYANK P

BFS: VAIBHAV G, KANAK S, JADHAV D

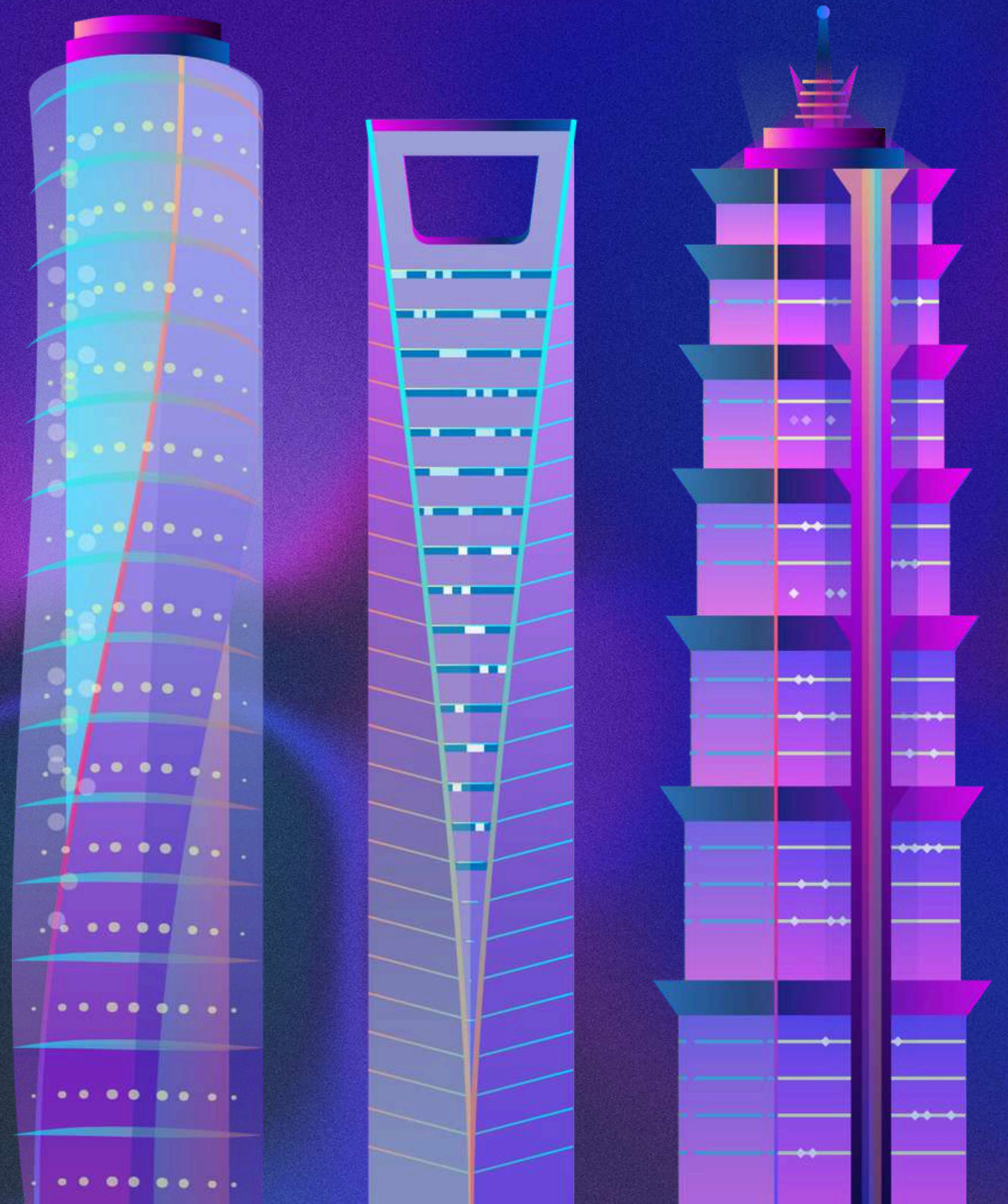
DFS: SANDEEP K, AKSHAYA G, ANUBHAV J, KHUSHI S

DJIKSTRA'S ALGO: ALOK K, GOPAL M, LOVERAJ

PROJECT: SRIVATSA

PRESENTERS: KANAK S, AKSHITA D, SRUSTHTI,
HIMANSHU R AND DARSH J

AND LAST BUT DEFINITELY NOT THE LEAST, OUR
MENTORS NITISH AND MANISH WITHOUT WHICH THIS
WOULD HAVE NEVER BEEN POSSIBLE.



THANK



YOU!